

Student 31 **SHAIK SHOUIB AKTHAR** 192312208RB

4 / 4 submitted

Q1. Arrays, Array Applications and Structures**CODE**

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[1000];
    int freq[256] = {0};
    int i;
    if (scanf("%s", str) != 1) {
        return 0;
    }
    for (i = 0; str[i] != '\0'; i++) {
        freq[(unsigned char)str[i]]++;
    }
    for (i = 0; str[i] != '\0'; i++) {
        if (freq[(unsigned char)str[i]] == 1) {
            printf("%c\n", str[i]);
            return 0;
        }
    }

    return 0;
}
```

INPUT

aabbcd

OUTPUT

(no output)

CODE

```
#include <stdio.h>

int searchRotatedArray(int arr[], int n, int target) {
    int low = 0, high = n - 1;

    while (low ≤ high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target)
            return mid;

        if (arr[low] ≤ arr[mid]) {

            if (target ≥ arr[low] && target < arr[mid])
                high = mid - 1;
            else
                low = mid + 1;
        } else {

            if (target > arr[mid] && target ≤ arr[high])
                low = mid + 1;
            else
                high = mid - 1;
        }
    }

    return -1;
}

int main() {
    int n;

    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int target;

    scanf("%d", &target);

    int result = searchRotatedArray(arr, n, target);

    if (result ≠ -1)
        printf("Package ID %d found at index %d.\n", target, result);
    else
        printf("Package ID %d not found in the array.\n", target);

    return 0;
}
```

INPUT

```
7
4 5 6 7 0 1 2
0
```

OUTPUT

(no output)

Q3. Arrays, Array Applications and Structures**CODE**

```
#include <stdio.h>
#include <stdlib.h>

struct Interval {
    int start;
    int end;
};

int compare(const void *a, const void *b) {
    struct Interval *i1 = (struct Interval *)a;
    struct Interval *i2 = (struct Interval *)b;
    return i1->start - i2->start;
}

int main() {
    int n;

    scanf("%d", &n);

    struct Interval intervals[n];

    for (int i = 0; i < n; i++) {
        scanf("%d %d", &intervals[i].start, &intervals[i].end);
    }

    qsort(intervals, n, sizeof(struct Interval), compare);

    struct Interval merged[n];
    int mergedCount = 0;

    merged[0] = intervals[0];
    mergedCount = 1;

    for (int i = 1; i < n; i++) {
        struct Interval *last = &merged[mergedCount - 1];

        if (intervals[i].start ≤ last->end) {
            if (intervals[i].end > last->end) {
                last->end = intervals[i].end;
            }
        } else {
            merged[mergedCount].start = intervals[i].start;
            merged[mergedCount].end = intervals[i].end;
            mergedCount++;
        }
    }

    printf("Merged Meeting Intervals:\n");
    for (int i = 0; i < mergedCount; i++) {
        printf("[%d, %d]\n", merged[i].start, merged[i].end);
    }
}
```

```
    return 0;  
}
```

INPUT

```
4  
1 3  
2 6  
8 10  
15 18
```

OUTPUT

(no output)

Q4. Arrays, Array Applications and Structures**CODE**

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[1000];
    int count[256];
    int i, j, k;

    // Initialize count array to 0 manually to be safe
    for (i = 0; i < 256; i++) {
        count[i] = 0;
    }

    // Read input
    if (fgets(str, sizeof(str), stdin) == NULL) {
        return 0;
    }

    // Remove newline
    int len = strlen(str);
    if (len > 0 && str[len - 1] == '\n') {
        str[len - 1] = '\0';
        len--;
    }

    // Count frequencies
    for (i = 0; i < len; i++) {
        int val = (int)str[i];
        count[val] = count[val] + 1;
    }

    // Build result string
    char result[1000];
    int resIdx = 0;

    // Loop 256 times to find the highest frequency remaining
    for (int pass = 0; pass < 256; pass++) {
        int maxF = 0;
        int bestChar = -1;

        for (i = 0; i < 256; i++) {
            if (count[i] > maxF) {
                maxF = count[i];
                bestChar = i;
            }
        }

        // If no more characters found, exit loop
        if (bestChar == -1) {
            break;
        }

        // Add the most frequent character to result
        for (k = 0; k < maxF; k++) {
            result[resIdx] = (char)bestChar;
            resIdx++;
        }

        // Set frequency to 0 so we don't pick it again
        count[bestChar] = 0;
    }
    result[resIdx] = '\0';
}
```

```
printf("%s\n", result);

return 0;
}
```

INPUT

tree

OUTPUT

(no output)

Student 32 **Srikanth S** 192514055RB

0 / 4 submitted

Q1. Arrays, Array Applications and Structures

— Not Submitted —

Q2. Arrays, Array Applications and Structures

— Not Submitted —

Q3. Arrays, Array Applications and Structures

— Not Submitted —

Q4. Arrays, Array Applications and Structures

— Not Submitted —

Student 33 **Surya.m** 192312476RB

0 / 4 submitted

Q1. Arrays, Array Applications and Structures

— Not Submitted —

Q2. Arrays, Array Applications and Structures

— Not Submitted —

Q3. Arrays, Array Applications and Structures

— Not Submitted —

Q4. Arrays, Array Applications and Structures

— Not Submitted —